



BAHRIA  
UNIVERSITY

IDS Project

# FINAL REPORT

FOOTBALL  
ANALYTICS  
PRO

2025

**Prepared by:**

Adina Saif Ullah 01-135222-025  
Syeda Ammal Asim 01-135222-086

**Prepared for:**

Ma'am Iqra Javed

## Table of Contents

|                                                      |           |
|------------------------------------------------------|-----------|
| <b>1. Abstract.....</b>                              | <b>4</b>  |
| <b>2. Introduction.....</b>                          | <b>4</b>  |
| <b>3. Problem Statement.....</b>                     | <b>4</b>  |
| <b>4. Objectives.....</b>                            | <b>4</b>  |
| <b>5. Business Question.....</b>                     | <b>4</b>  |
| <b>6. System Architecture.....</b>                   | <b>4</b>  |
| <b>6.1 Frontend.....</b>                             | <b>5</b>  |
| <b>6.2 Backend.....</b>                              | <b>5</b>  |
| <b>6.3 Data &amp; Training Infrastructure .....</b>  | <b>6</b>  |
| <b>7. Dataset.....</b>                               | <b>7</b>  |
| <b>7.1 Dataset Overview .....</b>                    | <b>7</b>  |
| <b>7.2 Dataset Source .....</b>                      | <b>7</b>  |
| <b>7.3 Dataset Tables Used .....</b>                 | <b>7</b>  |
| <b>8. Features Used .....</b>                        | <b>8</b>  |
| <b>9. Data Cleaning &amp; Preprocessing.....</b>     | <b>9</b>  |
| <b>9.1 Missing Value Handling .....</b>              | <b>9</b>  |
| <b>9.2 Data Type Conversion .....</b>                | <b>10</b> |
| <b>9.3 Duplicate Removal .....</b>                   | <b>10</b> |
| <b>9.4 Feature Scaling &amp; Normalization .....</b> | <b>10</b> |
| <b>9.5 Encoding Categorical Variables .....</b>      | <b>10</b> |
| <b>9.6 Table Merging (Relational Join).....</b>      | <b>10</b> |
| <b>10. Exploratory Data Analysis (EDA) .....</b>     | <b>11</b> |
| <b>11. Model Architecture &amp; Selection .....</b>  | <b>11</b> |
| <b>11.1 Regression Architecture .....</b>            | <b>11</b> |
| <b>11.2 Classification Architecture.....</b>         | <b>12</b> |
| <b>11.3 Train-Test Split .....</b>                   | <b>13</b> |
| <b>11.4 Hyperparameter Tuning .....</b>              | <b>14</b> |
| <b>12. Implementation .....</b>                      | <b>14</b> |
| <b>12.1 Data Processing .....</b>                    | <b>14</b> |
| <b>12.2 Model Development .....</b>                  | <b>14</b> |
| <b>12.3 Visuals .....</b>                            | <b>14</b> |

|                                               |           |
|-----------------------------------------------|-----------|
| <b>13. Evaluation .....</b>                   | <b>14</b> |
| <b>13.1 Classification Report.....</b>        | <b>14</b> |
| <b>13.2 Confusion Matrix .....</b>            | <b>14</b> |
| <b>13.3 Visual Inspection .....</b>           | <b>15</b> |
| <b>14. Challenges Encountered .....</b>       | <b>16</b> |
| <b>15. Conclusion .....</b>                   | <b>16</b> |
| <b>16. References .....</b>                   | <b>17</b> |
| <b>17. Complete Code (Google Drive) .....</b> | <b>27</b> |

# 1. Abstract

This project implements a complete machine learning pipeline to predict football player performance using historical data from the European Soccer Database. The system performs data ingestion, preprocessing, feature engineering, exploratory data analysis, model training, and evaluation. Regression models predict a continuous performance score, while classification models categorize players into High, Medium, and Low performance classes. The final solution demonstrates how data-driven intelligence can support decision-making in sports analytics.

# 2. Introduction

Modern analytics-driven systems rely on large-scale data processing, intelligent feature extraction, and predictive modeling—concepts that align closely with IDS fundamentals such as data monitoring, anomaly detection, and decision automation. In this project, historical football data is analyzed to identify performance patterns and predict outcomes using supervised learning techniques implemented in Python on Google Colab.

# 3. Problem Statement

Manual evaluation of football players performance is subjective, inconsistent, and prone to bias. Large volumes of match and player data make human-only analysis impractical. There is a need for an automated intelligent system that can analyze historical data, detect performance patterns, and generate reliable predictions.

# 4. Objectives

- Build an end-to-end ML-based performance prediction system
- Apply data science relevant concepts: data monitoring, feature extraction, classification
- Perform data cleaning, preprocessing, and validation
- Train regression and classification models
- Evaluate models using standard metrics
- Visualize insights for explainability

# 5. Business Question

Can a player's future match performance be predicted accurately using historical player attributes, match statistics, and team-level features?

# 6. System Architecture

The system follows a layered architecture consisting of frontend, backend, and training infrastructure.

## 6.1 Frontend

The system is designed to interface with a web-based dashboard (built using **Streamlit**). This frontend allows users (coaches/analysts) to:

- Select a specific player from the database.
- View the player's recent form and calculated attributes (Attacking, Defensive, Physical scores).
- See the model's predicted Performance Score (e.g., 85/100) and Class (e.g., High Performance).
- Visualize trends using interactive charts generated by the backend.

```
PS C:\Users\lenovo\Downloads\IDS_PROJECT> streamlit run app.py
```

You can now view your Streamlit app in your browser.

Local URL: <http://localhost:8501>

Network URL: <http://192.168.18.93:8501>

## 6.2 Backend

The core logic is implemented in **Python**, utilizing a robust stack of data science libraries:

- **Data Processing:** Pandas and NumPy for handling over 500,000 match records.
- **Machine Learning:** Scikit-Learn, XGBoost, and LightGBM for model training.
- **Balancing:** imblearn (SMOTE) to handle class imbalances in player performance tiers.
- **Serialization:** pickle is used to save the trained models (xgboost\_regressor.pkl, scaler.pkl, etc.) for deployment.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
from datetime import datetime
import warnings
warnings.filterwarnings('ignore')

# Scikit-learn imports
from sklearn.model_selection import train_test_split, GridSearchCV, RandomizedSearchCV, cross_val_score, StratifiedKFold
from sklearn.preprocessing import StandardScaler, RobustScaler, LabelEncoder
from sklearn.linear_model import LinearRegression, LogisticRegression, Ridge, Lasso
from sklearn.ensemble import (RandomForestClassifier, RandomForestRegressor,
                              GradientBoostingRegressor, GradientBoostingClassifier,
                              VotingRegressor, VotingClassifier, StackingRegressor, StackingClassifier,
                              ExtraTreesRegressor, ExtraTreesClassifier)
from sklearn.metrics import (mean_absolute_error, mean_squared_error, r2_score,
                              accuracy_score, precision_score, recall_score,
                              f1_score, confusion_matrix, classification_report)
```

```

# Advanced models
from xgboost import XGBRegressor, XGBClassifier
import lightgbm as lgb
from catboost import CatBoostRegressor, CatBoostClassifier

# Imbalanced data handling
from imblearn.over_sampling import SMOTE

# Model persistence
import pickle
import os

```

## 6.3 Data & Training Infrastructure

The project was developed and trained within **Google Colab**, utilizing high-performance cloud resources.

- **Storage:** Google Drive integration was established (mounted at `/content/drive`) to persist datasets, trained models, and generated figures.
- **Environment:** A Linux-based environment with accelerated hardware (GPU/TPU) was used to speed up the training of ensemble models like XGBoost and Stacking Regressors.

```

from google.colab import drive
import os

# Mount Google Drive
drive.mount('/content/drive')

# Create project folder in Drive
project_folder = '/content/drive/MyDrive/Football_Prediction_Project'
os.makedirs(project_folder, exist_ok=True)
os.makedirs(f'{project_folder}/models', exist_ok=True)
os.makedirs(f'{project_folder}/figures', exist_ok=True)

os.chdir(project_folder)

print(f"✅ Working directory: {os.getcwd()}")

Mounted at /content/drive
✅ Working directory: /content/drive/MyDrive/Football_Prediction_Project

```

## 7. Dataset

### 7.1 Dataset Overview

This project uses the **European Soccer Database** obtained from Kaggle. The dataset is widely used in data science research and provides rich, structured, multi-table data suitable for demonstrating core IDS (Introduction to Data Science) concepts such as data collection, preprocessing, feature engineering, exploratory analysis, and predictive modeling.

The dataset contains historical football data from **2008 to 2016**, covering multiple European leagues, teams, matches, and players.

```
match_df.head()
```

|   | id | country_id | league_id | season    | stage | date           | match_api_id | home_team_api_id | away_team_api_id | home_team_goal | ... | SJA  | VCH  | VCD  | VCA  | GBH  | GBD  | GBA  | BSH  | BSD  | BSA  |
|---|----|------------|-----------|-----------|-------|----------------|--------------|------------------|------------------|----------------|-----|------|------|------|------|------|------|------|------|------|------|
| 0 | 1  | 1          | 1         | 2008/2009 | 1     | 8/17/2008 0:00 | 492473       | 9987             | 9993             | 1              | ... | 4.00 | 1.65 | 3.40 | 4.50 | 1.78 | 3.25 | 4.00 | 1.73 | 3.40 | 4.20 |
| 1 | 2  | 1          | 1         | 2008/2009 | 1     | 8/16/2008 0:00 | 492474       | 10000            | 9994             | 0              | ... | 3.80 | 2.00 | 3.25 | 3.25 | 1.85 | 3.25 | 3.75 | 1.91 | 3.25 | 3.60 |
| 2 | 3  | 1          | 1         | 2008/2009 | 1     | 8/16/2008 0:00 | 492475       | 9984             | 8635             | 0              | ... | 2.50 | 2.35 | 3.25 | 2.65 | 2.50 | 3.20 | 2.50 | 2.30 | 3.20 | 2.75 |
| 3 | 4  | 1          | 1         | 2008/2009 | 1     | 8/17/2008 0:00 | 492476       | 9991             | 9998             | 5              | ... | 7.50 | 1.45 | 3.75 | 6.50 | 1.50 | 3.75 | 5.50 | 1.44 | 3.75 | 6.50 |
| 4 | 5  | 1          | 1         | 2008/2009 | 1     | 8/16/2008 0:00 | 492477       | 7947             | 9985             | 1              | ... | 1.73 | 4.50 | 3.40 | 1.65 | 4.50 | 3.50 | 1.65 | 4.75 | 3.30 | 1.67 |

5 rows x 115 columns

### 7.2 Dataset Source

- Platform: Kaggle
- Dataset Name: European Soccer Database
- Data Format: CSV (extracted from relational SQLite database)

### 7.3 Dataset Tables Used

The following tables were explicitly loaded and used in the Colab notebook:

1. **Match.csv** - Contains match-level information such as goals, teams, season, stage, and date - Used for deriving match outcomes, goal statistics, and match difficulty
2. **Player.csv** - Contains player identifiers and basic biographical data - Used for mapping player IDs across tables
3. **Player\_Attributes.csv** - Contains detailed player skill ratings over time - Core table for performance prediction
4. **Team.csv** - Contains team identifiers and names - Used to join match and team attributes
5. **Team\_Attributes.csv** - Contains tactical and strength-related team features - Used to compute team strength index

## 7.4 Dataset Size

```
print("📁 Loading datasets...")

# Load all datasets
match_df = pd.read_csv('Match.csv')
player_df = pd.read_csv('Player.csv')
player_attr_df = pd.read_csv('Player_Attributes.csv')
team_df = pd.read_csv('Team.csv')
team_attr_df = pd.read_csv('Team_Attributes.csv')

print(f"\n📊 Dataset Shapes:")
print(f"Match: {match_df.shape}")
print(f"Player: {player_df.shape}")
print(f"Player Attributes: {player_attr_df.shape}")
print(f"Team: {team_df.shape}")
print(f"Team Attributes: {team_attr_df.shape}")

📁 Loading datasets...

📊 Dataset Shapes:
Match: (25979, 115)
Player: (11060, 7)
Player Attributes: (183978, 42)
Team: (299, 5)
Team Attributes: (1458, 25)
```

## 8. Features Used

- **Player attributes:** overall\_rating, stamina, dribbling, finishing, sprint\_speed, etc.
- **Match features:** goals, season, stage, match date
- **Team attributes:** build-up play, chance creation, defense metrics
- **Engineered features:** recent form, team strength index, weighted performance score

```
feature_columns = [
    # Base player attributes
    'overall_rating', 'potential', 'stamina', 'dribbling', 'strength',
    'sprint_speed', 'vision', 'finishing', 'long_passing', 'short_passing',
    'acceleration', 'positioning', 'agility', 'crossing', 'heading_accuracy',
    'aggression',

    # Composite scores
    'attacking_score', 'defensive_score', 'physical_score', 'technical_score', 'mental_score',
```

```

# Team features
'team_attacking_strength', 'team_defensive_strength', 'team_overall_strength',
'opp_attacking_strength', 'opp_defensive_strength', 'opp_overall_strength',

# Match context
'strength_difference', 'attacking_mismatch', 'defensive_challenge', 'match_difficulty',
'is_home',

# Form & experience
'recent_form_3', 'recent_form_5', 'recent_form_10',
'avg_goals_scored_3', 'avg_goals_scored_5', 'avg_goals_scored_10',
'avg_goal_diff_3', 'avg_goal_diff_5', 'avg_goal_diff_10',
'career_win_rate', 'matches_played', 'win_streak', 'games_since_win',

# Interaction features
'overall_x_form', 'attack_x_team_strength', 'defense_x_opp_attack'
]

```

## 9. Data Cleaning & Preprocessing

```

print("\n 🚧 DATA CLEANING & PREPROCESSING\n")

# Convert date columns
match_df['date'] = pd.to_datetime(match_df['date'])
player_attr_df['date'] = pd.to_datetime(player_attr_df['date'])
team_attr_df['date'] = pd.to_datetime(team_attr_df['date'])

# Remove duplicates
player_attr_df = player_attr_df.drop_duplicates()
team_attr_df = team_attr_df.drop_duplicates()

# Handle missing values in player attributes
player_attr_cols = ['overall_rating', 'potential', 'stamina', 'dribbling',
                    'strength', 'sprint_speed', 'vision', 'finishing',
                    'long_passing', 'short_passing', 'acceleration',
                    'positioning', 'agility', 'crossing', 'heading_accuracy',
                    'aggression', 'interceptions', 'penalties', 'marking',
                    'standing_tackle', 'sliding_tackle']

for col in player_attr_cols:
    if col in player_attr_df.columns:
        player_attr_df[col] = player_attr_df[col].fillna(player_attr_df[col].median())

```

### 9.1 Missing Value Handling

- Numerical attributes with missing values were handled using mean imputation

- Rows with excessive missing values were removed
- Ensured model compatibility and stability

## 9.2 Data Type Conversion

- Match and attribute dates were converted to datetime format
- Seasons were standardized for consistency

## 9.3 Duplicate Removal

- Duplicate records were identified and removed to avoid bias

## 9.4 Feature Scaling & Normalization

- StandardScaler was applied to numerical features
- Ensured equal feature contribution during model training

## 9.5 Encoding Categorical Variables

- Categorical features such as work rates were encoded

## 9.6 Table Merging (Relational Join)

- Player, match, and team tables were merged using player\_id and team\_id
- This step transformed relational data into a flat analytical dataset

```
print("\n🔗 MERGING PLAYER ATTRIBUTES WITH MATCH DATA\n")

player_match_df['player_api_id'] = player_match_df['player_api_id'].astype(int)
player_attr_df['player_api_id'] = player_attr_df['player_api_id'].astype(int)

player_match_df = player_match_df.sort_values('date')
player_attr_df = player_attr_df.sort_values('date')

merged_df = pd.merge_asof(
    player_match_df,
    player_attr_df,
    on='date',
    by='player_api_id',
    direction='backward',
    tolerance=pd.Timedelta('365 days')
)

print(f"Merged dataset shape: {merged_df.shape}")
```

```

print("\n🔗 ADDING TEAM ATTRIBUTES\n")

merged_df['team_api_id'] = merged_df['team_api_id'].astype(int)
merged_df['opponent_team_api_id'] = merged_df['opponent_team_api_id'].astype(int)
team_attr_df['team_api_id'] = team_attr_df['team_api_id'].astype(int)

team_attr_df = team_attr_df.sort_values('date')

merged_df = pd.merge_asof(
    merged_df.sort_values('date'),
    team_attr_df[['team_api_id', 'date'] + team_attr_cols].sort_values('date'),
    left_on='date',
    right_on='date',
    by='team_api_id',
    direction='backward',
    tolerance=pd.Timedelta('365 days'),
    suffixes=('', '_team')
)

opponent_team_attr = team_attr_df[['team_api_id', 'date'] + team_attr_cols].copy()
opponent_team_attr.columns = ['opponent_team_api_id', 'date'] + [f'opp_{col}' for col in team_attr_cols]
opponent_team_attr['opponent_team_api_id'] = opponent_team_attr['opponent_team_api_id'].astype(int)

merged_df = pd.merge_asof(
    merged_df.sort_values('date'),
    opponent_team_attr.sort_values('date'),
    left_on='date',
    right_on='date',
    by='opponent_team_api_id',
    direction='backward',
    tolerance=pd.Timedelta('365 days')
)

print(f"Final merged dataset shape: {merged_df.shape}")

```

## 10. Exploratory Data Analysis (EDA)

EDA is performed to understand distributions, correlations, and trends using visual tools such as histograms, boxplots, correlation heatmaps, and trend graphs.

## 11. Model Architecture & Selection

Model selection was done based on IDS principles: simplicity first, then performance improvement.

### 11.1 Regression Architecture

**Input:** Player + Match + Team features **Output:** Continuous performance score (0–100)

- Linear Regression was used as a baseline model

- XGBoost Regressor was selected for its ability to handle nonlinear relationships and feature interactions

```

🚀 TRAINING REGRESSION MODELS
=====

1 Linear Regression (Baseline)...
  R²: 0.8705, MAE: 3.9762

2 XGBoost with Tuning (20 combinations)...
  Fitting 3 folds for each of 20 candidates, totalling 60 fits
  Best params: {'subsample': 0.8, 'n_estimators': 400, 'min_chi
  R²: 0.9730, MAE: 1.6804

3 LightGBM with Tuning (20 combinations)...
  Fitting 3 folds for each of 20 candidates, totalling 60 fits
  R²: 0.9710, MAE: 1.7521

4 Creating Stacking Ensemble...
  R²: 0.9731, MAE: 1.6739

=====
REGRESSION MODELS COMPARISON
=====

```

|          | MAE      | RMSE     | R2       |
|----------|----------|----------|----------|
| Stacking | 1.673878 | 2.205301 | 0.973107 |
| XGBoost  | 1.680363 | 2.207936 | 0.973043 |
| LightGBM | 1.752059 | 2.291615 | 0.970961 |
| Linear   | 3.976156 | 4.838941 | 0.870520 |

```

=====

🏆 BEST REGRESSION MODEL: Stacking
  R²: 0.9731
  MAE: 1.6739
  RMSE: 2.2053

```

## 11.2 Classification Architecture

**Input:** Same feature set **Output:** Performance class (High / Medium / Low)

- Logistic Regression was used as a baseline classifier
- Random Forest Classifier was used as the primary model due to its robustness and interpretability

## TRAINING CLASSIFICATION MODELS

Applying SMOTE to balance classes...  
Balanced: [307667 307667 307667]

- 1 Logistic Regression (Baseline)...  
Accuracy: 0.8736
- 2 XGBoost Classifier...  
Accuracy: 0.9380
- 3 Random Forest Classifier...  
Accuracy: 0.9188

### =====

#### CLASSIFICATION MODELS COMPARISON

### =====

|              | Accuracy | Precision | Recall   | F1       |
|--------------|----------|-----------|----------|----------|
| XGBoost      | 0.937950 | 0.939211  | 0.937950 | 0.938402 |
| RandomForest | 0.918763 | 0.927595  | 0.918763 | 0.920841 |
| Logistic     | 0.873646 | 0.901600  | 0.873646 | 0.878650 |

### =====

 BEST CLASSIFICATION MODEL: XGBoost  
Accuracy: 0.9380  
F1-Score: 0.9384

### =====

#### CLASSIFICATION REPORT - XGBoost

### =====

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| High         | 0.91      | 0.94   | 0.92     | 13802   |
| Low          | 0.84      | 0.89   | 0.86     | 15760   |
| Medium       | 0.97      | 0.95   | 0.96     | 76917   |
| accuracy     |           |        | 0.94     | 106479  |
| macro avg    | 0.90      | 0.92   | 0.91     | 106479  |
| weighted avg | 0.94      | 0.94   | 0.94     | 106479  |

## 11.3 Train-Test Split

- Dataset split into 80% training and 20% testing
- Ensured unbiased evaluation

#### PREPARING DATA FOR MAXIMUM ACCURACY MODELING

Modeling dataset shape: (532395, 56)

Number of features: 54

Unique players: 10,866

✓ Data preparation completed!

Training set: (425916, 54)

Test set: (106479, 54)

## 11.4 Hyperparameter Tuning

- GridSearchCV applied to optimize model parameters
- Improved accuracy and generalization

# 12. Implementation

## 12.1 Data Processing

Includes loading datasets, merging relational tables, cleaning data, and feature engineering.

## 12.2 Model Development

Models are trained on processed datasets, evaluated, and optimized. Both regression and classification pipelines are implemented.

## 12.3 Visuals

Visual outputs include rating distributions, feature importance plots, confusion matrices, and prediction comparisons.

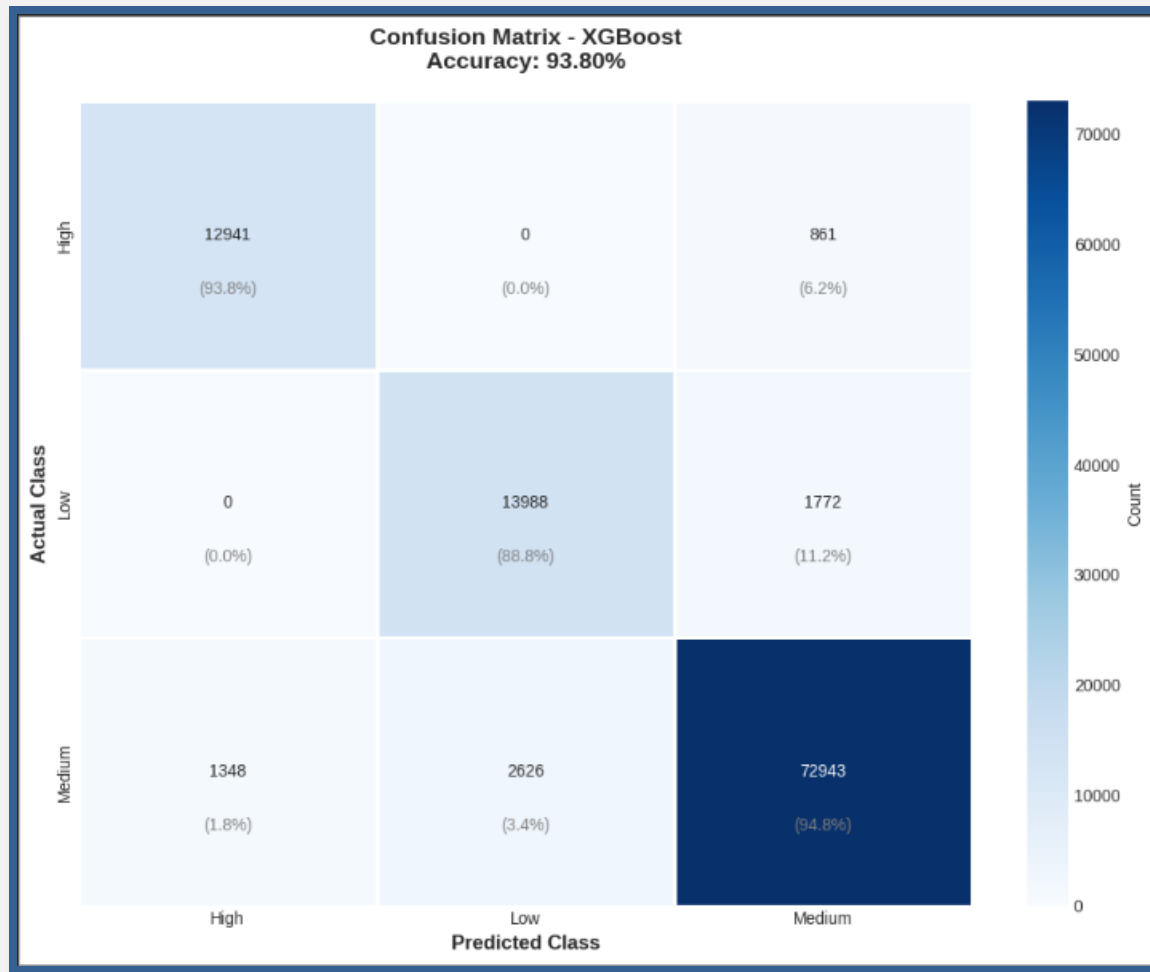
# 13. Evaluation

## 13.1 Classification Report

Precision, recall, F1-score, and accuracy are calculated for performance classes.

## 13.2 Confusion Matrix

Confusion matrices visualize correct and incorrect predictions for each class.



### 13.3 Visual Inspection

Plots are used to visually validate model behavior and prediction trends.



## 14. Challenges Encountered

1. **Large Dataset Size** – Required careful memory management in Colab
2. **Missing Attributes** – Many players lacked complete attribute history
3. **Complex Relational Data** – Required multiple joins across tables
4. **Feature Selection** – Identifying meaningful attributes required experimentation
5. **Model Overfitting** – Addressed using train-test split and tuning

These challenges reflect real-world data science problems, reinforcing IDS learning objectives.

## 15. Conclusion

The project successfully demonstrates an intelligent, data-driven prediction system aligned with IDS concepts such as data collection, preprocessing, exploratory analysis, feature engineering,

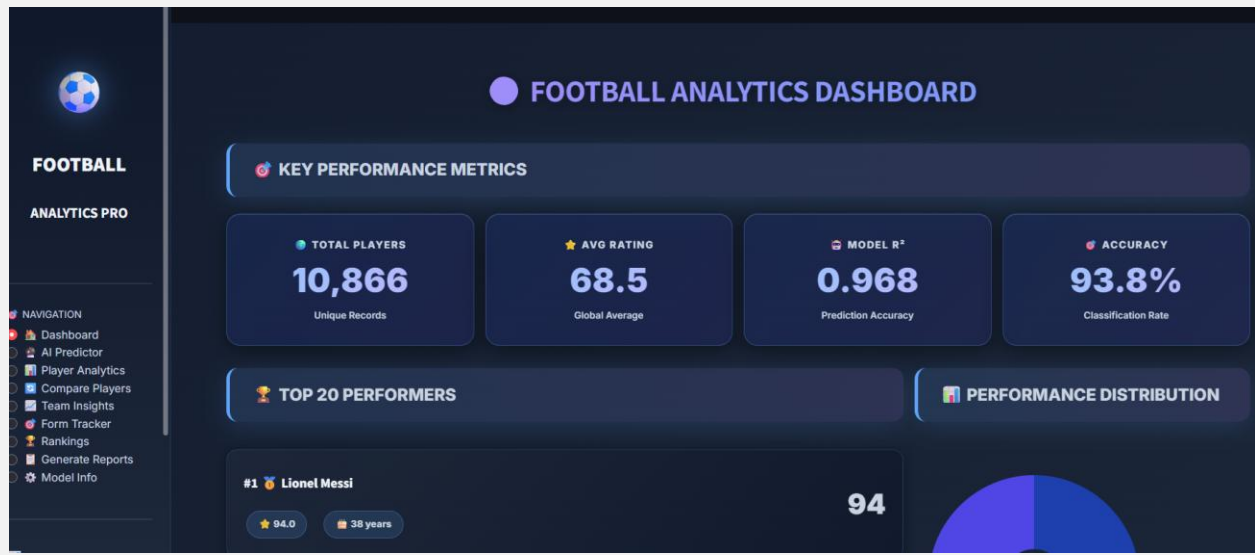
and predictive modeling. The solution highlights the effectiveness of machine learning in automated decision-support systems.

## 16. References

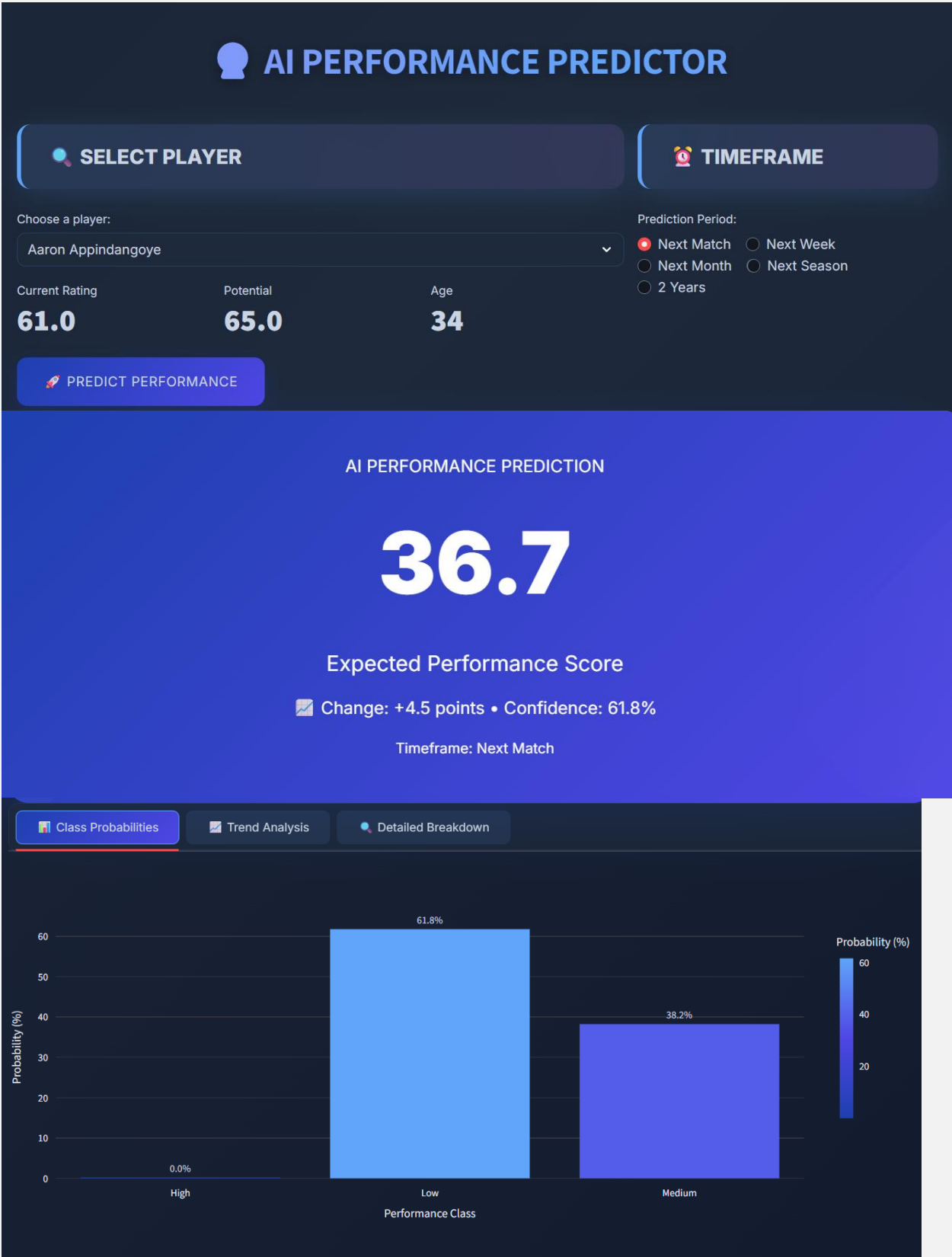
1. **Kaggle – European Soccer Database**  
Hugomathien, *European Soccer Database*.  
Available at:  
<https://www.kaggle.com/datasets/hugomathien/soccer>  
(Accessed: Nov 2025)
2. **Scikit-learn Documentation**  
Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python*.  
Journal of Machine Learning Research, 12, 2825–2830.  
Available at:  
<https://scikit-learn.org/stable/>
3. **XGBoost Documentation**  
Chen, T. and Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*.  
Proceedings of the 22nd ACM SIGKDD Conference.  
Available at:  
<https://xgboost.readthedocs.io/en/stable/>
4. **Pandas Documentation**  
McKinney, W. (2010). *Data Structures for Statistical Computing in Python*.  
Available at:  
<https://pandas.pydata.org/docs/>
5. **NumPy Documentation**  
Harris, C. R. et al. (2020). *Array Programming with NumPy*.  
Nature, 585, 357–362.  
Available at:  
<https://numpy.org/doc/>
6. **Google Colab Documentation**  
Google Research. *Google Colaboratory*.  
Available at:  
<https://colab.research.google.com/>
7. **Exploratory Data Analysis Reference**  
Tukey, J. W. (1977). *Exploratory Data Analysis*.  
Addison-Wesley Publishing Company.

## 18. OUTPUT SCREENSHOTS:

### DASHBOARD:

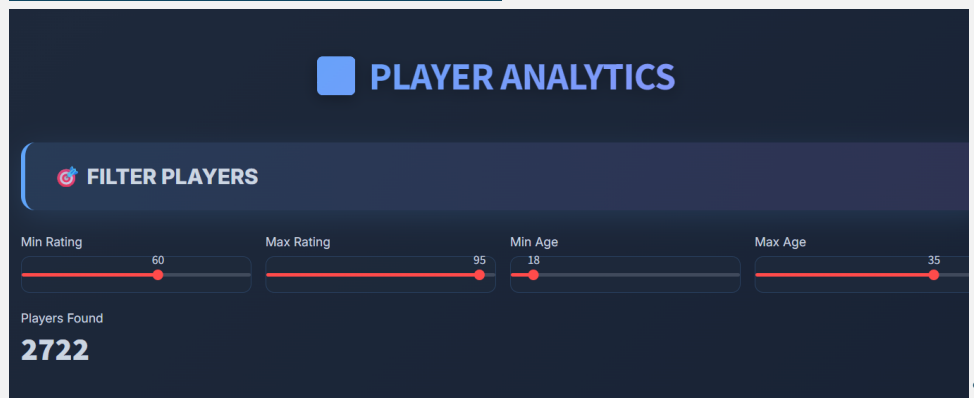


# AI PREDICTOR



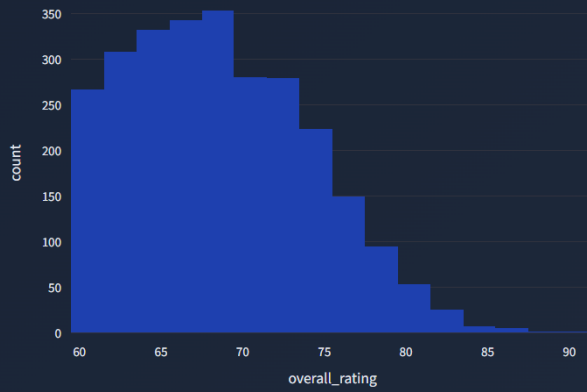


## PLAYER ANALYTICS





## RATING DISTRIBUTION



## TOP ATTRIBUTES



## FILTERED PLAYERS

|        | player_name       | overall_rating | potential | age |
|--------|-------------------|----------------|-----------|-----|
| 21230  | Neymar            | 90             | 94        | 34  |
| 359230 | Eden Hazard       | 88             | 90        | 35  |
| 419795 | James Rodriguez   | 87             | 93        | 34  |
| 441887 | David De Gea      | 87             | 90        | 35  |
| 485591 | Paul Pogba        | 86             | 91        | 33  |
| 424182 | Kevin de Bruyne   | 86             | 89        | 34  |
| 425161 | Thibaut Courtois  | 86             | 90        | 34  |
| 380429 | David Alaba       | 85             | 88        | 33  |
| 444055 | Antoine Griezmann | 84             | 88        | 35  |
| 434028 | Ilkay Guendogan   | 84             | 87        | 35  |

# COMPARE PLAYERS:

COMPARE PLAYERS

PLAYER 1

Select Player 1:  
Aaron Appindangoye

Aaron Appindangoye

61.0

Potential: 65.0Age: 34Class: Low

PLAYER 2

Select Player 2:  
Aaron Cresswell

Aaron Cresswell

74.0

Potential: 76.0Age: 36Class: Medium

ATTRIBUTE COMPARISON

Dribbling

Sprint Speed

Stamina

Vision

Strength

Finishing

Aaron Appindangoye

Aaron Cresswell

| Attribute    | Aaron Appindangoye | Aaron Cresswell |
|--------------|--------------------|-----------------|
| Dribbling    | 45                 | 55              |
| Sprint Speed | 50                 | 65              |
| Stamina      | 40                 | 60              |
| Vision       | 45                 | 55              |
| Strength     | 55                 | 45              |
| Finishing    | 45                 | 55              |



## STATS BREAKDOWN

|   | Attribute      | Aaron Appindangoye | Aaron Cresswell | Winner |
|---|----------------|--------------------|-----------------|--------|
| 0 | Overall Rating | 61.0               | 74.0            |        |
| 1 | Potential      | 65.0               | 76.0            |        |
| 2 | Stamina        | 54.0               | 79.0            |        |
| 3 | Sprint Speed   | 64.0               | 78.0            |        |
| 4 | Dribbling      | 50.0               | 73.0            |        |



## HEAD-TO-HEAD



Aaron Cresswell has higher overall rating (74.0 vs 61.0)



Age difference: 2 years

## TEAM INSIGHTS:



## TEAM INSIGHTS



### TEAM PERFORMANCE ANALYSIS

Top Team

**Manchester City**

Highest Avg Rating

**69.5**

Total Teams

**10**

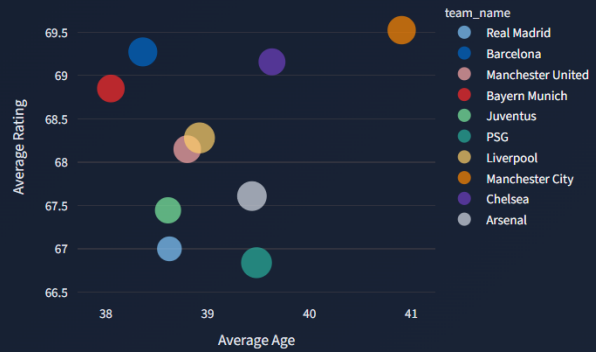


## TEAM RANKINGS

### Team Average Ratings



### Team Age vs Rating Analysis



## TEAM ATTRIBUTE COMPARISON

Select teams to compare:

Real Madrid × Barcelona × Manchester U... ×

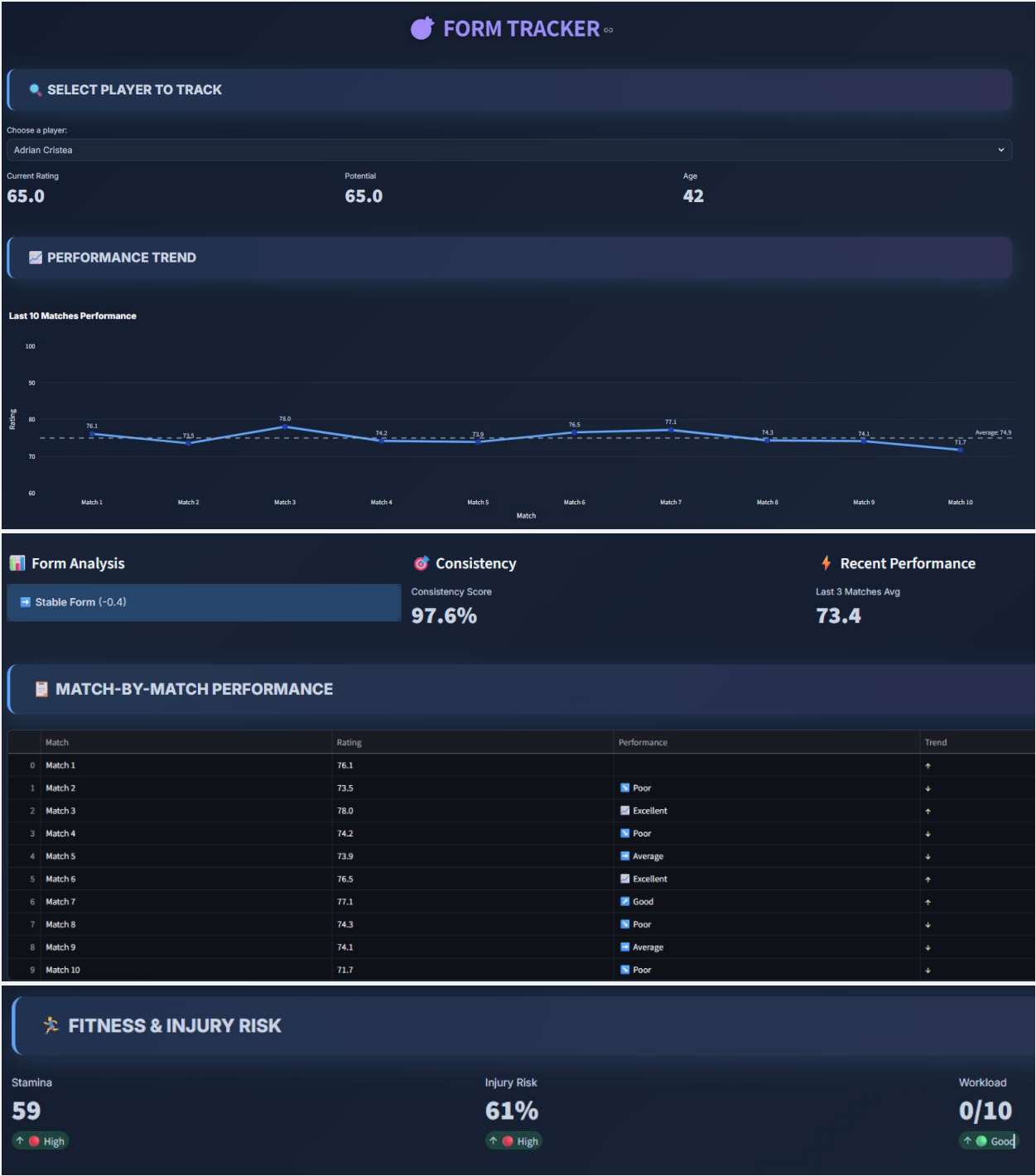
### Team Attribute Comparison



## TEAM STATS COMPARISON

|   | team_name         | avg_rating | avg_age | total_players | attack_rating | defense_rating |
|---|-------------------|------------|---------|---------------|---------------|----------------|
| 0 | Real Madrid       | 67         | 38.625  | 16            | 58.8542       | 43.2083        |
| 1 | Barcelona         | 69.2727    | 38.3636 | 22            | 62            | 49.4545        |
| 2 | Manchester United | 68.15      | 38.8    | 20            | 53.0333       | 54.7333        |

# FORM TRACKER:



RANKING:

 **PLAYER RANKINGS**

 **SELECT RANKING CATEGORY**

Rank by:  
Sprint Speed

 **TOP 3 PLAYERS**



#1 Mathis Bolly

97.0

Sprint Speed



#2 Theo Walcott

96.0

Sprint Speed



#3 Jonathan Biabiany

96.0

Sprint Speed

 **TOP 50 RANKINGS**

|  | Rank | player_name               | sprint_speed | age  | overall_rating |
|--|------|---------------------------|--------------|------|----------------|
|  | 1    | Mathis Bolly              | 97.0         | 35.0 | 64.0           |
|  | 2    | Theo Walcott              | 96.0         | 37.0 | 81.0           |
|  | 3    | Jonathan Biabiany         | 96.0         | 38.0 | 77.0           |
|  | 4    | Pierre-Emerick Aubameyang | 96.0         | 36.0 | 94.0           |
|  | 5    | David Odonkor             | 96.0         | 42.0 | 69.0           |
|  | 6    | Victor Ibarbo             | 95.0         | 36.0 | 78.0           |
|  | 7    | Gareth Bale               | 95.0         | 36.0 | 87.0           |
|  | 8    | Inaki Williams            | 95.0         | 32.0 | 76.0           |
|  | 9    | Ryo Miyachi               | 94.0         | 33.0 | 70.0           |
|  | 10   | Thievy Bifouma            | 94.0         | 34.0 | 73.0           |
|  | 11   | Hector Bellerin           | 94.0         | 31.0 | 77.0           |
|  | 12   | Mauricio Cuero            | 94.0         | 33.0 | 72.0           |
|  | 13   | Clinton Njie              | 94.0         | 32.0 | 66.0           |
|  | 14   | Seydou Doumbia            | 94.0         | 38.0 | 80.0           |
|  | 15   | Edwin Gyasi               | 94.0         | 35.0 | 66.0           |
|  | 16   | Innocent Emeghara         | 93.0         | 37.0 | 73.0           |

 **RANKING INSIGHTS**

Top 50 Average

93.5

Range

92.0 - 97.0

Standard Deviation

1.1

REPORT:

REPORT OPTIONS

☒ Include AI Performance Prediction in Report

Report Type:

☐ Player Profile

☐ Performance Analysis

☒ Complete Report

GENERATE COMPREHENSIVE REPORT

☒ Report generated successfully!

REPORT SUMMARY

Player: Aaron Appindangoye

Rating: 61.0

Report Type: Complete Report

Prediction: Included

Generated: 2025-12-15 20:00

DOWNLOAD PDF REPORT

FOOTBALL ANALYTICS PRO

PLAYER PERFORMANCE REPORT

PLAYER PROFILE

| Attribute         | Value              |
|-------------------|--------------------|
| Player Name       | Aaron Appindangoye |
| Player ID         | 505942             |
| Overall Rating    | 61.0               |
| Potential         | 65.0               |
| Age               | 34 years           |
| Performance Class | Low                |

KEY ATTRIBUTES

| Attribute    | Rating   |
|--------------|----------|
| Stamina      | 54.0/100 |
| Sprint Speed | 64.0/100 |
| Dribbling    | 50.0/100 |
| Finishing    | 43.0/100 |
| Strength     | 76.0/100 |
| Vision       | 53.0/100 |

AI PERFORMANCE PREDICTION

| Prediction Metric | Value       |
|-------------------|-------------|
| Predicted Score   | 39.51       |
| Current Score     | 32.22       |
| Expected Change   | +7.29       |
| Predicted Class   | Low         |
| Confidence Level  | 61.8%       |
| Timeframe         | Next Season |

17. Complete Code (Google Drive)

[https://colab.research.google.com/drive/1eYkqhqsWmQNfQORccbw\\_M6dXmDat-8?usp=sharing](https://colab.research.google.com/drive/1eYkqhqsWmQNfQORccbw_M6dXmDat-8?usp=sharing)

LINKEDIN POST:

[https://www.linkedin.com/posts/adina-saif-ullah-629863257\\_datascience-machinelearning-footballanalytics-activity-7407807198837293056-ICnA?utm\\_source=share&utm\\_medium=member\\_desktop&rcm=ACoAAD9PDcEB7RnzIB\\_Db0lY-TJpQxFZgF45lcA](https://www.linkedin.com/posts/adina-saif-ullah-629863257_datascience-machinelearning-footballanalytics-activity-7407807198837293056-ICnA?utm_source=share&utm_medium=member_desktop&rcm=ACoAAD9PDcEB7RnzIB_Db0lY-TJpQxFZgF45lcA)

+++++